Software Design Specification

Inventory and Sales Tracker for Small Shops

Version 1.0 | October 2026
Prepared by: Syed Mohib, University of Karachi

1. Introduction

1.1 Purpose

This Software Design Specification (SDS) describes how the Inventory and Sales Tracker will be designed and built to satisfy the requirements in the Software Requirements Specification (SRS). It covers the architecture, modules, database, API, key algorithms, user interface, and security design, and serves as the guide for implementation and testing.

1.2 Scope

The design covers a web-based system for small shops with a React frontend, a Node.js/Express backend, and a PostgreSQL database. It supports products and stock, purchases, sales and billing, customer credit (udhaar), and reporting for two roles: Owner and Cashier.

1.3 Design Goals

- Correctness: stock and money values must always stay consistent.
- Simplicity: a cashier should create a bill in a few steps.
- Maintainability: separate layers so each part can be changed independently.
- Security: role-based access and protected credentials.

2. System Architecture

2.1 Architectural Style

The system uses a three-tier client-server architecture. The browser runs the React single-page application, which talks to a stateless REST API over HTTPS using JSON. The API applies business rules and is the only component that accesses the database.

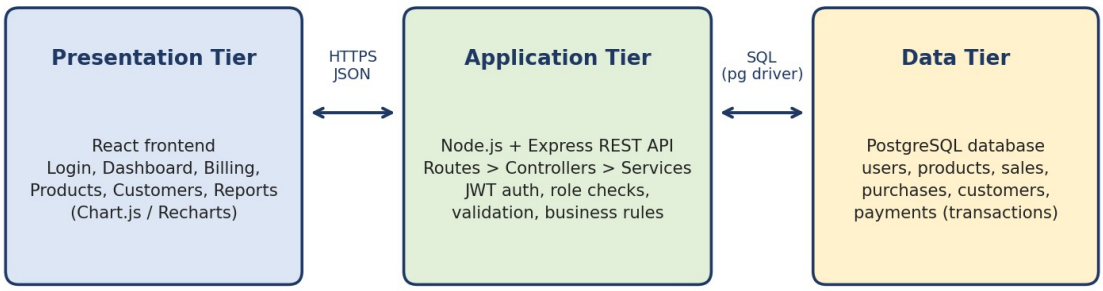


Figure 1: Three-tier architecture

2.2 Technology Stack

Layer	Technology and purpose
Frontend	React with React Router for pages; Axios for API calls; Recharts or Chart.js for charts; CSS framework for responsive layout.

Layer	Technology and purpose
Backend	Node.js with Express; bcrypt for password hashing; jsonwebtoken for sessions; express-validator for input checks.
Database	PostgreSQL accessed through the pg driver (or an ORM such as Prisma/Sequelize).
Tools	Git and GitHub for version control; Postman for API testing; VS Code as the editor.

2.3 Module Decomposition

Module	Responsibility
Auth module	Login, token creation and verification, role checks, user management.
Product module	CRUD for products and categories, search, low-stock list.
Purchase module	Supplier records and purchase entry that increases stock.
Sales module	Bill creation, stock deduction, receipt data, sale voiding.
Customer module	Customer records, credit balances, payments, ledger.
Report module	Daily, weekly, and monthly sales, profit, and best-seller queries.

2.4 Backend Layering

Each module follows the same layers: a route maps a URL to a controller; the controller reads and validates the request; a service holds the business logic and transactions; and a data-access layer runs SQL queries. Shared middleware handles authentication, authorization, validation, and error responses.

3. Database Design

3.1 Entity Relationships

A category has many products. A supplier has many purchases, and a purchase has many purchase items. A customer has many sales and many payments, and a sale has many sale items. Each sale item and purchase item refers to one product. Users record sales, purchases, and payments.

3.2 Table Definitions

Table	Columns	Keys and constraints
users	id, name, username, password_hash, role, is_active	PK id; username unique; role in (owner, cashier)
categories	id, name	PK id; name unique
products	id, name, category_id, cost_price, sale_price, stock_qty, reorder_level, is_active	PK id; FK category_id; stock_qty >= 0; prices >= 0
suppliers	id, name, phone	PK id
purchases	id, supplier_id, user_id, purchase_date, total	PK id; FK supplier_id, user_id
purchase_items	id, purchase_id, product_id, qty, unit_cost	PK id; FK purchase_id, product_id; qty > 0
customers	id, name, phone, balance	PK id; balance numeric(12,2)

Table	Columns	Keys and constraints
sales	id, customer_id, user_id, sale_date, total, paid_amount, status	PK id; FK customer_id (nullable), user_id; status in (completed, voided)
sale_items	id, sale_id, product_id, qty, unit_price, unit_cost	PK id; FK sale_id, product_id; qty > 0
payments	id, customer_id, user_id, amount, payment_date	PK id; FK customer_id, user_id; amount > 0

3.3 Design Decisions

- unit_price and unit_cost are copied into sale_items, so old profit figures do not change when prices are updated later.
- Products are deactivated instead of deleted when they appear in past sales.
- Money columns use NUMERIC(10,2) to avoid rounding errors; indexes are added on sale_date, product name, and customer_id.

4. API Design

All endpoints are under /api, exchange JSON, and require a valid JWT except login. "Owner" means owner-only; "Both" means owner and cashier.

Method	Endpoint	Description	Access
POST	/auth/login	Verify credentials and return a token	Public
GET, POST	/products	List (with search) or add products	Both / Owner
PUT, DELETE	/products/:id	Update or deactivate a product	Owner
GET	/products/low-stock	Products at or below reorder level	Both
GET, POST	/categories, /suppliers	List or add categories and suppliers	Owner
POST, GET	/purchases	Record or list purchases	Owner
POST	/sales	Create a sale (transactional)	Both
GET	/sales/:id	Get sale and receipt data	Both
PATCH	/sales/:id/void	Void a sale and restore stock	Owner
GET, POST	/customers	List or add customers	Both
POST	/customers/:id/payments	Record a credit payment	Both
GET	/customers/:id/ledger	Customer transaction history	Both
GET	/reports/sales?period=	Daily, weekly, or monthly sales	Owner
GET	/reports/profit, /reports/top-products	Profit and best-selling items	Owner

5. Detailed Design of Key Processes

5.1 Create Sale

The sale is saved inside one database transaction so that either everything is stored or nothing is.

1. Start a transaction (BEGIN).
2. For each item, read the product row with a lock (SELECT ... FOR UPDATE) and confirm stock_qty is enough; otherwise roll back and return an error.
3. Insert the sale record and one sale_item per product, copying the current sale price and cost price.
4. Reduce stock_qty of each product by the sold quantity.
5. If paid_amount is less than total, add the difference to the customer balance (a customer is required).
6. Commit the transaction and return the sale with receipt data.

5.2 Low-Stock Alert

After every sale and on dashboard load, the system queries products where stock_qty <= reorder_level and is_active is true, and shows them in an alert panel with product name, current stock, and reorder level.

5.3 Profit Calculation

Profit for a period is the sum of (unit_price - unit_cost) x qty over all completed sale items in that period. Voided sales are excluded.

5.4 Void Sale

Only the owner can void a sale. In one transaction the system marks the sale as voided, adds the quantities back to stock, and subtracts any unpaid amount of that sale from the customer balance.

6. User Interface Design

Screen	Main elements
Login	Username, password, error message, login button.
Dashboard	Today's sales and profit (owner), low-stock alerts, recent sales, quick link to billing.
Billing (POS)	Product search, item list with quantity and price, total, customer selector, amount paid, change or credit, confirm and print.
Products	Searchable table with category filter, add/edit form, stock and reorder level columns.
Purchases	Supplier selector, item entry, total, purchase history.
Customers	Customer list with balances, payment form, ledger view.
Reports	Period selector, sales and profit charts, best-selling products, export button.

The layout is responsive, uses large buttons on the billing screen, and shows clear validation messages next to each field.

7. Security Design

- Passwords are hashed with bcrypt and never stored or returned in plain text.
- The server issues a JWT with an expiry time; every protected route verifies it and checks the user role.
- All queries use parameters (prepared statements) to prevent SQL injection; inputs are validated and output is escaped to prevent XSS.
- HTTPS is used in deployment, and database credentials are kept in environment variables.

8. Testing and Deployment

Unit tests cover business rules such as stock deduction, credit balance, and profit. API tests in Postman verify each endpoint and role restriction, and user acceptance tests use real shop scenarios. For deployment, the React build is served as static files, the Express server runs under a process manager, and PostgreSQL is backed up daily.